

Concurrencia

Prácticas 1 y 2

Grado en Ingeniería Informática/ Grado en Matemáticas e Informática/ 2ble. grado en Ing. Informática y ADE
Convocatoria de Semestre feb–jun 2025–2026

Normas

- Os recordamos que **todas** las prácticas entregadas pasan por un proceso automático de detección de copias.

Calendario de entregas

29 de mayo de 2026 23:59:59	Fecha límite de entrega de la práctica 1
5 de junio de 2026 23:59:59	Fecha límite de entrega de la práctica 2

1. Control de un mercado

En esta práctica se va a implementar la lógica para controlar un mercado electrónico sencillo, donde vendedores y compradores intercambian productos por dinero.

Un agente vendedor ofrece productos para su venta – en la práctica siempre el mismo producto para simplificar – y establece un precio *mínimo* (*min*) aceptable y el tiempo máximo en que la oferta de venta está disponible. De manera similar un agente comprador se ofrece a comprar un producto, especificando el precio *máximo* aceptable (*max*), y hasta cuándo está vigente la oferta de comprar.

Una venta solo puede llevarse a cabo si el precio mínimo del vendedor es menor o igual que el precio máximo del comprador y si la venta se realiza antes que se haya agotado el tiempo dispuesto.

El precio fijado para una compra sera $(min + max)/2$ calculado con aritmética de enteros en Java.

Como simplificación la práctica asume que cada vendedor (o comprador) no ofrece (o compra) varios productos a la vez, la cantidad siempre es 1.

1.1. Observando el mercado

Para poder observar las transacciones en el mercado el API a implementar permite comprobar el éxito de una oferta de vender o comprar. Esta operación devuelve 0 (indicando que el producto no fue vendido), o la cantidad de dinero ganado (venta) o gastado (compra).

También existen operaciones que funcionan como alertas si durante el intervalo de tiempo actual el precio mínimo acordado es menor o igual que un límite (información útil para un comprador), o si el precio máximo acordado es mayor o igual que un límite (información útil para un vendedor).

1.2. Diseño

En nuestro programa concurrente, cada *agente* (vendedor y/o comprador) va a ser simulado por un proceso. Además de los procesos *agente* incluiremos un proceso encargado de modelar el paso de tiempo (el *reloj*).

En la figura 1 se muestra la arquitectura del programa concurrente en forma de grafo de procesos y recursos. Como se puede observar, los procesos se comunicarán a través de un recurso compartido implementado como una instancia de Mercado. Las secciones 1.2.1 y 1.2.2 describen los procesos y el recurso compartido.

1.2.1. Código de los procesos

La figura 2 muestra la interfaz en Java del recurso compartido. Los aspectos más relevantes para entender los procesos son:

- Los agentes utilizan venta para declarar su voluntad de vender un producto; la oferta está en vigor hasta que se aceptada o bien ha pasado el tiempo especificado, que es igual que un número de llamadas a la operación tick del proceso reloj. La operación devuelve un identificador único

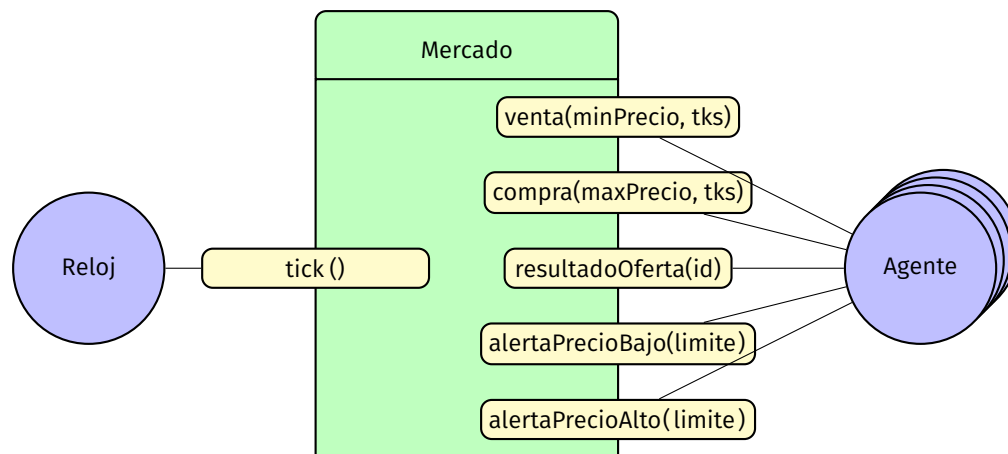


Figura 1: Grafo de procesos/recursos.

```

public interface Mercado {
    public int venta(int minPrecio, int tiempo);
    public int compra(int maxPrecio, int tiempo);
    public int resultadoOferta(int id);
    public void alertaPrecioBajo(int limite);
    public void alertaPrecioAlto(int limite);
    public void tick();
}

```

Figura 2: Interfaz en Java del recurso compartido.

(un entero) para la oferta de venta. **En la búsqueda de una compra compatible para una venta se deben priorizar las compras que especifiquen un precio mayor, y si dos compras tienen el mismo precio, se priorizará la compra con el identificador menor.**

- Los agentes utilizan compra para declarar su voluntad de comprar un producto. La operación devuelve un identificador único para la oferta de compra. **En la búsqueda de una venta compatible con una compra se deben priorizar las ventas que especifiquen un precio menor, y si dos ventas tienen el mismo precio, se priorizará la venta con el identificador menor.**
- El proceso del reloj genera los *ticks* que marcan el paso de tiempo. Si pasa demasiado tiempo esto puede hacer que ofertas de vender o comprar un producto queden invalidadas. Llamamos intervalo de tiempo *actual* a lo que ocurre entre la última llamada a tick y hasta la siguiente.
- Para comprobar el estado de una oferta de vender o comprar un producto un agente utiliza resultadoOferta. Este método bloquea hasta que el producto ha sido vendido (comprado) o hasta que expira su tiempo de venta (o compra). Este método tiene como parámetro el identificador de oferta devuelto por las operaciones venta o compra. El método devuelve 0 si la oferta no fue aceptada, y si fue aceptada, devuelve el precio acordado.
- Un agente (principalmente un comprador) puede comprobar que el precio acordado en una compraventa (en el intervalo de tiempo actual) es menor o igual que un límite mediante la operación alertaPrecioBajo. Una llamada a la operación bloquea hasta que se realiza una compraventa que cumple con dicho límite.
- Un agente (principalmente un vendedor) puede comprobar que el precio acordado en una compraventa (en el intervalo de tiempo actual) es mayor o igual que un límite mediante la operación alertaPrecioAlto. Una llamada a la operación bloquea hasta que se realiza una compraventa que cumple con dicho límite.

1.2.2. Especificación formal del recurso

La especificación formal del recurso compartido se muestra en las figuras 3 y 4. Las siguientes líneas ofrecen aclaraciones sobre la representación escogida:

- El tipo Tks representa el número de *ticks* del reloj: números naturales > 0 .
- El tipo Id representa el identificador de una oferta, un número natural.
- El tipo Oferta representa una oferta de venta o compra. Es una tupla con los elementos
 - p: el precio deseado,
 - t: el tiempo que queda para realizar la oferta, y
 - d: el dinero gastado o ganado al realizar la compra/venta.
- El dominio, y por tanto **self**, es una tupla con los elementos
 - ventas: una función parcial de identificadores a ofertas de venta,
 - compras: una función parcial de identificadores a ofertas de compra,
 - mx: el precio máximo acordado en el intervalo de tiempo actual,
 - mi: el precio mínimo acordado en el intervalo de tiempo actual,
 - cn: un contador para asignar identificadores a ofertas.
- $\text{MatchV}(m,v)$ es el conjunto de compras dentro del mercado m que son compatibles con la venta v . Para ser compatible una compra c tiene que ser *activa* (0 dinero gastado/pagado, y queda tiempo) y tener un precio mayor o igual que el precio de la venta v . Además no puede existir otra compra activa c' activa que especifique un precio mayor que el de c , o si los precios son iguales, que tenga una identidad menor que la identidad de c . La definición implica que el conjunto es vacío (si no hay una compra compatible) o es un conjunto con una única compra.
- $\text{MatchC}(m,c)$ es el conjunto de ventas dentro el mercado m que son compatibles con la compra c , y se define análogamente a $\text{MatchV}(m,v)$.
- Las *precondiciones* sólo están para documentar el protocolo que los agentes van a respetar. Por lo tanto no es necesario comprobar las precondiciones en vuestra implementación.

1.2.3. Implementación del recurso

En la corrección de las prácticas se va a considerar la eficiencia de la implementación. Por ejemplo, puede ser una buena idea tener las ofertas de comprar y vender ordenadas (según precio e identificador) mediante alguna estructura de datos auxiliar. Por supuesto, a efectos de acceso en exclusión mutua, esas estructuras auxiliares también formarían parte del estado del recurso.

C-TAD Mercado**ACCIÓN** Venta: $\mathbb{N}^+[e] \times \text{Tks}[e] \times \text{Id}[s]$ **ACCIÓN** Compra: $\mathbb{N}^+[e] \times \text{Tks}[e] \times \text{Id}[s]$ **ACCIÓN** ResultadoOferta: $\text{Id}[e] \times \mathbb{N}[s]$ **ACCIÓN** AlertaPrecioBajo: $\mathbb{N}[e]$ **ACCIÓN** AlertaPrecioAlto: $\mathbb{N}[e]$ **ACCIÓN** Tick**DONDE****TIPO** Id = $\mathbb{N}$ **TIPO** Tks = $\mathbb{N}$ **TIPO** Oferta = $(p: \mathbb{N}^+ \times t: \text{Tks} \times d: \mathbb{N})$ **SEMÁNTICA****DOMINIO****TIPO** Mercado = $(\text{ventas: Id} \mapsto \text{Oferta} \times \text{compras: Id} \mapsto \text{Oferta} \times \text{mx: } \mathbb{Z} \times \text{mi: } \mathbb{Z}, \text{cn: } \mathbb{N})$ **INICIAL self** = $(\emptyset, \emptyset, -\infty, \infty, 0)$ **CPRE: cierto**Venta(p, tks, **result**)**POST:** **self**^{pre} = (ventas, compras, mx, mi, cn) $\wedge$ **result** = cn $\wedge$ **self**.cn = cn+1 $\wedge [(\text{tks} = 0 \vee \text{MatchV}(\text{self}^{\text{pre}}, p) = \emptyset) \Rightarrow (v = (p, \text{tks}, 0)$ $\wedge$ **self**.compras = compras $\wedge$ **self**.ventas = ventas $\oplus \{ \text{cn} \mapsto v \}$ $\wedge$ **self**.mx = mx $\wedge$ **self**.mi = mi)] $\wedge [(\text{tks} > 0 \wedge \exists! (i, c) \bullet \text{MatchV}(\text{self}^{\text{pre}}, p) = \{(i, c)\}) \Rightarrow (\text{precio} = (c.p + p)/2$ $\wedge v = (p, \text{tks}, \text{precio})$ $\wedge c' = (c.p, c.t, \text{precio})$ $\wedge$ **self**.ventas = ventas $\oplus \{ \text{cn} \mapsto v \}$ $\wedge$ **self**.compras = compras $\oplus \{ i \mapsto c' \}$ $\wedge$ **self**.mx = **max**(mx, precio) $\wedge$ **self**.mi = **min**(mi, precio))]**CPRE: cierto**Compra(p, tks, **result**)**POST:** **self**^{pre} = (ventas, compras, mx, mi, cn) $\wedge$ **result** = cn $\wedge$ **self**.cn = cn+1 $\wedge [(\text{tks} = 0 \vee \text{MatchC}(\text{self}^{\text{pre}}, p) = \emptyset) \Rightarrow (c = (p, \text{tks}, 0)$ $\wedge$ **self**.ventas = ventas $\wedge$ **self**.compras = compras $\oplus \{ \text{cn} \mapsto c \}$ $\wedge$ **self**.mx = mx $\wedge$ **self**.mi = mi)] $\wedge [(\text{tks} > 0 \wedge \exists! (i, v) \bullet \text{MatchC}(\text{self}^{\text{pre}}, p) = \{(i, v)\}) \Rightarrow (\text{precio} = (v.p + p)/2$ $\wedge c = (p, \text{tks}, \text{precio})$ $\wedge v' = (v.p, v.t, \text{precio})$ $\wedge$ **self**.compras = compras $\oplus \{ \text{cn} \mapsto c \}$ $\wedge$ **self**.ventas = ventas $\oplus \{ i \mapsto v' \}$ $\wedge$ **self**.mx = **max**(mx, precio) $\wedge$ **self**.mi = **min**(mi, precio))]

Figura 3: Especificación formal del recurso compartido (parte 1).

PRE: $id \in \text{dom self.compras} \vee id \in \text{dom self.ventas}$
CPRE: $(c = \text{self}^{\text{pre}}.\text{compras}(id) \wedge (c.d > 0 \vee c.t = 0)) \vee (v = \text{self}^{\text{pre}}.\text{ventas}(id) \wedge (v.d > 0 \vee v.t = 0))$
 ResultadoOferta(id, result)
POST: $(c = \text{self}^{\text{pre}}.\text{compras}(id) \Rightarrow \text{result} = c.d) \wedge (v = \text{self}^{\text{pre}}.\text{ventas}(id) \Rightarrow \text{result} = v.d)$
 $\wedge \text{self} = \text{self}^{\text{pre}}$

CPRE: $\text{limite} \geq \text{self}^{\text{pre}}.\text{mi}$
 AlertaPrecioBajo(limite)
POST: $\text{self} = \text{self}^{\text{pre}}$

CPRE: $\text{limite} \leq \text{self}^{\text{pre}}.\text{mx}$
 AlertaPrecioAlto(limite)
POST: $\text{self} = \text{self}^{\text{pre}}$

CPRE: cierto
 Tick
POST: $\text{self}.\text{mx} = -\infty \wedge \text{self}.\text{mi} = \infty \wedge \text{self}.\text{cn} = \text{self}^{\text{pre}}.\text{cn}$
 $\wedge \text{self}.\text{ventas} = \{ (i, v') \mid \exists (i, v) \in \text{self}^{\text{pre}}.\text{ventas} \bullet v'.p = v.p \wedge v'.d = v.d \wedge v'.t = \max(v.t-1, 0) \}$
 $\wedge \text{self}.\text{compras} = \{ (i, c') \mid \exists (i, c) \in \text{self}^{\text{pre}}.\text{compras} \bullet c'.p = c.p \wedge c'.d = c.d \wedge c'.t = \max(c.t-1, 0) \}$

DONDE
 $\text{MatchV}(m, p) \equiv$
 $\{ (i, c) \mid (i, c) \in m.\text{compras} \bullet c.d = 0 \wedge c.t > 0 \wedge p \leq c.p$
 $\wedge \neg \exists (i', c') \in m.\text{compras} \bullet (i' \neq i \wedge c'.d = 0 \wedge c'.t > 0$
 $\wedge (c'.p > c.p \vee (c'.p = c.p \wedge i' < i))) \}$
 $\text{MatchC}(m, p) \equiv$
 $\{ (i, v) \mid (i, v) \in m.\text{ventas} \bullet v.d = 0 \wedge v.t > 0 \wedge p \geq v.p$
 $\wedge \neg \exists (i', v') \in m.\text{ventas} \bullet (i' \neq i \wedge v'.d = 0 \wedge v'.t > 0$
 $\wedge (v'.p < v.p \vee (v'.p = v.p \wedge i' < i))) \}$

Figura 4: Especificación formal del recurso compartido (parte 2).

2. Prácticas

2.1. Primera práctica

La entrega consistirá en una implementación del recurso compartido en Java usando la clase `Monitor` de la librería `cclib`. La implementación a realizar debe estar contenida en un fichero llamado `MercadoMonitor.java` que implementará la interfaz `Mercado`.

2.2. Segunda práctica

La entrega consistirá en una implementación del recurso compartido en Java mediante paso de mensajes síncrono, usando la librería `JCSP`. La implementación deberá estar contenida en un fichero llamado `MercadoCSP.java` que implementará la interfaz `Mercado`.

3. Información general

La entrega de las prácticas se realizará **vía WWW** en la dirección

`http://deliverit.fi.upm.es`

Para facilitar la realización de la práctica están disponibles en el moodle de la asignatura varias unidades de compilación:

- `Mercado.java`: define la interfaz común a las distintas implementaciones del recurso compartido.
- `MercadoSim.java`: simulador de un mercado. Es un programa concurrente con procesos que siguen el diseño dado (es tarea vuestra cambiar la instanciación de `Mercado` para usar la implementación de monitores o la de paso de mensajes, buscad “new `Mercado...`”).
- `MercadoMonitor.java`: esqueleto para la práctica 1.
- `MercadoCSP.java`: esqueleto para la práctica 2, que incluirá la mayor parte del código *vernacular* de `JCSP` (se publicará más tarde).
- `cclib-0.4.9.jar`: biblioteca con mecanismos de concurrencia de la asignatura.
- `jcsp.jar`: biblioteca Communicating Sequential Processes for Java (`JCSP`) de la Universidad de Kent.

Por supuesto durante el desarrollo podéis cambiar el código que os entreguemos para hacer diferentes pruebas, depuración, etc., pero el código que entreguéis debe poder compilarse y ejecutar sin errores junto con el resto de los paquetes entregados **sin modificar estos últimos**. Podéis utilizar las bibliotecas estándar de Java y las bibliotecas auxiliares que están en el código de apoyo (se incluye `aedlib.jar`, biblioteca de la asignatura Algoritmos y Estructuras de Datos).

El programa de recepción de prácticas podrá rechazar entregas que:

- Tengan errores de compilación.
- Utilicen otras librerías o aparte de las estándar de Java y las que se han mencionado anteriormente.
- No estén suficientemente comentadas. Alrededor de un tercio de las líneas deben ser comentarios **significativos**. No se tomarán en consideración para su evaluación prácticas que tengan comentarios ficticios con el único propósito de rellenar espacio.
- No superen unas pruebas mínimas de ejecución, integradas en el sistema de entrega.